



Clojure (<http://clojure.org/>) is a dynamic, general purpose programming language that targets the Java Virtual Machine (JVM) and the Common Language Runtime (CLR). It is designed for work and play.

If you play around with Java, you are ready to run Clojure. If not, you will have to install the Java Runtime Environment (1.5+ is recommended), and Ant, if you want to build the Clojure sources. Download the current snapshot of Clojure from <https://github.com/clojure/clojure>, extract the archive, and fire off 'ant'. Once the build process is completed, you should have the file 'clojure.jar'. Fire up the Clojure 'prompt', as follows:

```
$ java -cp clojure.jar clojure.main
Clojure 1.3.0-master-SNAPSHOT
user=>
```

The Clojure prompt is now ready for your instructions. Let's try a few of them:

```
user=> (clojure-version)
"1.3.0-master-SNAPSHOT"

user=> (print)
nil

user=> (print "Hello world")
Hello worldnil

user=> (+ 1 1)
2
```

(*clojure-version*) is a built-in function that returns the current version of Clojure. This also demonstrates how you call

a function in Clojure. The *print* function does the obvious. The *nil* that you see appended at the end of the *Hello World* string is the return value of the function, *print*. In Clojure, 'nil' means nothing. If you are familiar with any language of the Lisp family, you have definitely guessed correctly that Clojure is a Lisp dialect. Since I will not be talking about the basics of Lisp programming, you will benefit from this article if you are just a little familiar with a Lisp dialect. On the other hand, you could use this article as a starting point to learn Clojure programming, and get a feel of what Lisp is all about.

REPL

The *user=>* prompt is what is known as the REPL, or the *read-eval-print-loop*. The REPL is your key to world domination. A little Java library called *JLine* (<http://jline.sourceforge.net/>) will help you in your quest by enhancing the editing support of the Clojure REPL. Once the *JLine* JAR is in your CLASSPATH, invoke Clojure REPL using *java -cp jline-0_9_5.jar:clojure.jar jline.ConsoleRunner clojure.main*. Let us now see how you can go about defining a simple function at the REPL:

```
user=> (defn hello [name]
(println "Hello, " name))
#'user/hello
user=> (hello "new guy")
Hello, new guy
nil
user=>
```